

Table des matières

Exercices 05 — Le code d'un pipeline robuste	1
Questions courtes	1
Questions pratiques	3
Diagnostics	3
Questions à développer	4
Mises en situation	4

Exercices 05 — Le code d'un pipeline robuste

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les exercices marqués [labo] portent sur le pipeline que vous venez d'écrire ; les autres vérifient que le cours est clair dans votre tête. Tout vient de la Théorie 05 et du Labo 05.

Le corrigé est dans `Corrige_05.md`. Répondez à l'énoncé en entier avant de l'ouvrir : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Ce qui se configure. Donnez deux valeurs du pipeline de Boréal qui appartiennent à `config.toml`, et deux qui n'y ont pas leur place bien qu'elles soient écrites dans le code.
2. Le mot de passe et le fichier versionné. Pourquoi un mot de passe ne se range-t-il pas dans `config.toml`, même en dernière ligne, même commenté ?
3. Un modèle sans valeur. Qu'est-ce que `.env.exemple` apporte, puisqu'il ne contient aucune valeur utilisable ?
4. La surcharge. Quelle variable d'environnement donnerait 5434 au port de la base de l'ERP, sans toucher au fichier de configuration ?
5. D'où vient un secret. Le code appelle `secret("ERP_MOT_DE_PASSE")`. Nommez trois endroits d'où cette valeur peut venir, selon l'environnement, sans que la ligne de code change.
6. Deux traces. Quelle est la différence de nature entre le fichier `pipeline_2026-09-15.log` et la table `pipeline.executions` ?
7. Le bon niveau. À quel niveau de journal inscrire : le début d'une étape, un essai raté suivi d'une reprise réussie, une étape qui s'arrête, le détail de chaque page d'API ?

8. La ligne écrite d'avance. La ligne de `pipeline.executions` s'écrit avant le travail, à l'état « en cours ». Qu'est-ce qu'on perdrait à l'écrire seulement à la fin ?
 9. Passager ou définitif. Classez ces réponses : 429, 401, 503, 404, un délai dépassé, une réponse JSON mal formée.
 10. Compter les attentes. Quatre essais et une attente de 2 secondes qui double : combien de temps d'attente en tout avant l'abandon, et combien d'attentes ?
 11. Pourquoi doubler. Pourquoi l'attente double-t-elle au lieu de rester à 2 secondes ?
 12. Ce que la reprise suppose. Pourquoi les reprises du Cours 05 supposent-elles l'idempotence du Cours 03 ?
 13. L'horloge donnée de l'extérieur. La fonction avec `reprises` reçoit un argument `dormir`. À quoi sert-il, et qui s'en sert ?
 14. Un test plus exigeant. Le test de la reprise vérifie `attentes == [2.0, 4.0, 8.0]`. En quoi est-ce plus fort que de vérifier que l'abandon a bien eu lieu ?
 15. Ce qu'on ne teste pas. Nommez deux chemins du pipeline que les trois tests ne couvrent pas, et dites comment ils se vérifient.
 16. Un projet jetable. Pourquoi les tests posent-ils `PIPELINE_PROJET` sur un dossier temporaire ?
 17. Deux commandes, deux résultats. `pytest` échoue sur `ModuleNotFoundError: No module named 'pipeline'`, mais `python -m pytest` passe. Pourquoi, et que corrige `pytest.ini` ?
 18. Le fichier qui rend un paquet exécutable. Quel fichier faut-il dans un dossier pour que `python -m <dossier>` l'exécute ?
 19. Le code de sortie. Le pipeline rend 0 ou 1. Qui lit cette valeur, et que se passerait-il s'il rendait toujours 0 ?
1. Hors du chemin hors ligne. Pourquoi `erp` et `meteo` ne font-elles pas partie de ce que lance tout ?
 2. L'ordre des étapes. `python -m pipeline raw bronze` est accepté par la porte d'entrée. Est-ce une bonne idée ?
 3. Un import déplacé. Dans `erp.py`, `import pyodbc` est écrit à l'intérieur de la fonction. Quelle serait la conséquence de le remonter en tête du module ?
 4. Dedans, dehors. Citez trois choses que le `Dockerfile` met dans l'image, et trois qu'il laisse dehors.
 5. Une version épinglée. Pourquoi `duckdb==1.5.5` plutôt que `duckdb` tout court ?
 6. Deux garde-fous. `.gitignore` et `.dockerignore` nomment tous deux `.env`. Contre quoi chacun protège-t-il ?
 7. Le fichier du lendemain. Le conteneur écrit `plateforme/journaux/pipeline_2026-09-16.log` alors qu'il est 20 h 30 le 15 chez vous. Pourquoi ?
 8. Le lignage qui traverse. Le lignage de la RAW garde `sources/erp/extraits/2023-01/VBAK.csv`. Qu'arriverait-il, au premier lancement dans le conteneur, s'il gardait `C:\...\sources\erp\extraits\2023-01\VBAK.csv` ?

9. La dernière instruction. Le module `bronze.py` lit le résultat de la dernière instruction des scripts SQL du Cours 02. Que se passe-t-il si quelqu'un ajoute une requête après le contrôle ?

Questions pratiques

1. [labo] Le mode bavard. Lancez le pipeline en mode bavard sur la seule étape du corpus, et dites quelles lignes apparaissent qui ne s'affichaient pas.
2. [labo] Le profil des étapes. Écrivez la requête qui donne, pour chaque étape, le nombre de passages, la durée moyenne et la durée la plus longue, de la plus lente à la plus rapide.
3. [labo] Les exécutions tuées. Écrivez la requête qui trouve les exécutions restées à l'état « en cours », puis fabriquez-en une pour vérifier qu'elle les voit.
4. [labo] Du tableau au journal. Une étape a échoué hier soir. Écrivez la requête qui rend son identifiant d'exécution et son message, puis dites comment retrouver le détail dans les fichiers de journal.
5. [labo] Un contrôle qui tombe. Faites perdre des lignes à la conversion du bronze en ajoutant un filtre au COPY de `sql/bronze_historien.sql`, et observez ce que le pipeline écrit dans les deux traces. Remettez le fichier en état ensuite.
6. [labo] Une station pour un seul lancement. Sans toucher à `config.toml`, faites lire au pipeline la station de météo 7060401, pour un lancement seulement.
7. [labo] Un quatrième test. Écrivez le test qui vérifie que la surcharge par l'environnement convertit bien `PIPELINE_ERP_PORT` en entier, et non en chaîne de caractères.
8. [labo] Le coût de la frontière. Mesurez l'écart de durée de l'étape `raw` entre un lancement sous Windows et un lancement dans le conteneur, sur un projet déjà chargé, et expliquez-le.

Diagnostics

1. Un collègue lance `python -m pipeline tout` et reçoit `FileNotFoundError: ...\\01_de\\pipeline\\config.toml` est introuvable. Le fichier existe pourtant. Que s'est-il passé ?
2. Le pipeline tourne toutes les nuits par un ordonnanceur. Depuis trois semaines, `pipeline.executions` montre `raw` en « réussi » avec 0 ligne chaque nuit, et personne ne s'en est inquiété. Le fournisseur a cessé de livrer les extraits il y a trois semaines. Qu'est-ce qui manque au pipeline ?
3. Une étape qui appelle une API répond en 40 minutes au lieu de 2, sans erreur. Les journaux montrent des lignes `WARNING ... essai 1 raté (HTTP 401), nouvel essai dans 2 s`. Qu'est-ce qui a été mal programmé ?
4. Le conteneur recharge les 1 572 fichiers à chaque lancement, alors que Windows dit « à jour ». Donnez deux causes possibles et la façon de trancher entre elles.

Questions à développer

1. Une étape de plus. Votre équipe veut ajouter une étape `garanties`, qui anonymise les données personnelles du portail avant de les livrer. Décrivez ce que vous écrivez, où, et ce que vous ajoutez aux tests, sans écrire le code.
2. La configuration par l'environnement seul. Un architecte propose de remplacer `config.toml` par des variables d'environnement seules, « comme le veut la méthode des douze facteurs », un ensemble de règles pour les applications déployées en continu, dont l'une veut que la configuration vienne de l'environnement et jamais du code. Répondez-lui : ce qu'il gagne, ce qu'il perd, et ce que vous proposez.
3. Les journaux au bout de deux ans. Le pipeline écrit ses journaux dans `plateforme/journaux/`, un fichier par jour, sans limite. Décrivez ce qui arrive au bout de deux ans, et deux façons de traiter la question.

Mises en situation

1. Chez Boréal, un ingénieur a validé un `.env` avec le mot de passe de la base de l'ERP il y a six mois, dans le dépôt interne. Il vient de s'en apercevoir et retire le fichier au commit du jour. Que lui répondez-vous, et dans quel ordre agissez-vous ?
2. L'équipe de l'usine veut lancer le pipeline elle-même, sans installer Python. On vous demande si l'image du Cours 05 suffit. Que vérifiez-vous avant de dire oui, et quelles limites annoncez-vous ?
3. Un pipeline voisin, écrit par une autre équipe, réessaie indéfiniment toute erreur, « pour ne jamais perdre une nuit ». Il n'a jamais échoué en un an. Que soupçonnez-vous, et comment le vérifiez-vous ?